

챕터 3. 예외 처리와 DTO

게시판의 기본 기능이 완료되었습니다. 오픈이는 뿌듯한 마음으로 선배에게 화면을 보여줬습니다.

선배는 목록과 상세 조회를 한 번씩 눌러보더니, 주소창 끝에 있는 게시글 번호를 슬쩍 **999**로 바꿨습니다.

선배 : "없는 번호예요. 이런 예외 상황도 테스트해 봤어요?"

엔터를 치자 응답 창에는 여전히 **200 성공**이 떴습니다. 하지만 응답 바디는 비어 있었습니다.

게시글이 없는 번호인데 왜 응답이 성공으로 나오지?

선배 : "데이터베이스에서 게시글을 못 찾으면 빈 값으로 응답이 오는데, 이 빈 값을 그대로 응답에 담아버리니 성공으로 응답이 뜨는 거예요. 만약 없는 게시글을 수정이나 삭제하려고 한다면 오류가 발생하겠죠. 서버에서는 이런 예외(Exception)가 발생했을 경우를 위해 미리 예외 처리를 해야 해요. 그래야 어떤 사유로 요청이 실패했는지 클라이언트에게 명확히 알려줄 수 있어요."

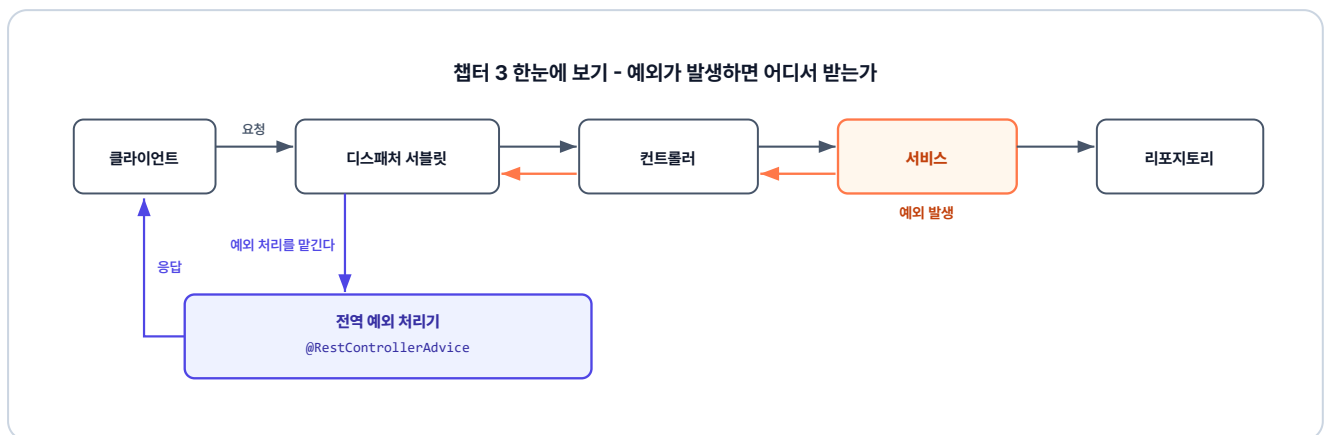


그림 3-1. 챕터 3의 실습 흐름

소스코드 준비

소스코드 준비에서 클론한 예제 저장소에서 이번 챕터 폴더로 이동합니다. 패키지 루트는 챕터 2와 같은 `com.metacoding.spring` 입니다.

터미널 챕터 3 폴더로 이동

```
cd spring-start/ch03
```

이번 챕터에서 새로 만들거나 고치는 파일은 다음과 같습니다. 나머지는 챕터 2 그대로입니다.

ch03 파일 구조

```
spring-start/ch03/src/main/java/com/metacoding/spring/
├── board/
│   ├── Board.java                # [작성] @Builder + 생성자 추가
│   ├── BoardController.java      # [작성] 요청·응답 타입 DTO로 교체
│   ├── BoardRequest.java         # [작성] 요청 DTO(SaveDTO/UpdateDTO) + toEntity()
│   ├── BoardRepository.java      # [작성] JpaRepository 인터페이스로 전환
│   ├── BoardResponse.java        # [작성] 응답 DTO(DTO/DetailDTO)
│   └── BoardService.java         # [작성] orElseThrow + DTO 반환
├── core/handler/
│   ├── ex/Exception400,401,403,404,500.java # [작성] 커스텀 예외
│   └── GlobalExceptionHandler.java          # [작성] 전역 예외 처리
```

챕터를 따라 코드를 채우고, 막히면 `spring-end` 의 완성 코드를 참고하세요.

3.1 요청 DTO

챕터 2의 컨트롤러는 요청을 **Board** 엔티티로 직접 받고, 응답할 때도 엔티티를 그대로 반환합니다. 하지만 실무에서는 엔티티를 요청이나 응답에 직접 사용하지 않습니다. 엔티티는 데이터베이스의 테이블 구조와 동일해서, 그대로 사용할 경우 응답에 불필요한 데이터가 노출되거나 테이블 구조가 변경될 때 예상치 못한 오류가 발생할 수 있기 때문입니다. 따라서 클라이언트와 주고받을 때는 필요한 데이터만 담아서 전달할 전용 객체를 사용해야 합니다.

3.1.1 DTO

계층 사이에서 데이터 전달만을 목적으로 하는 객체를 **DTO(Data Transfer Object)** 라고 합니다. 컨트롤러와 서비스처럼 서로 다른 계층 사이에서 필요한 값만 담아 전달합니다. DTO를 사용하면 요청 받을 값과 응답을 보낼 값을 필요한 형태로 정할 수 있습니다.

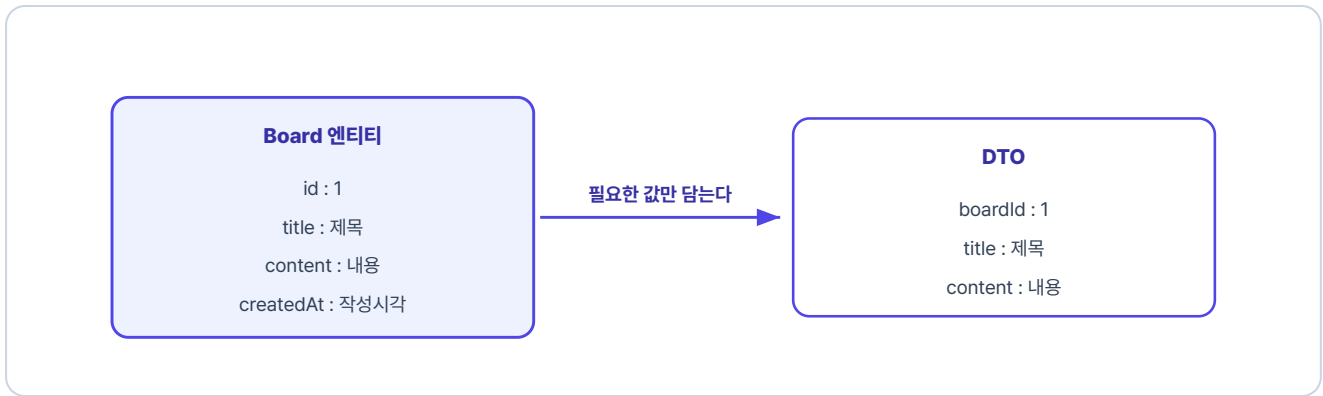


그림 3-2. 엔티티와 DTO가 담는 값

3.1.2 엔티티 생성자

데이터베이스에 저장되는 대상은 영속성 컨텍스트가 관리하는 엔티티입니다. 따라서 요청으로 받은 DTO를 데이터베이스에 저장하려면 엔티티로 변환하는 과정이 필요합니다. 이를 처리할 수 있도록 먼저 **Board** 클래스에 생성자를 추가합니다.

실습 1 board/Board.java. 빌더로 생성할 수 있게

```
@NoArgsConstructor // 기본 생성자 추가
@Data
@Entity
@Table(name = "board_tb")
public class Board {
    // id, title, content, createdAt 필드 (챕터 2와 같음)

    @Builder // 객체 생성 용도
    public Board(Integer id, String title, String content, LocalDateTime createdAt) {
        this.id = id;
        this.title = title;
        this.content = content;
        this.createdAt = createdAt;
    }
}
```

새로운 생성자를 직접 추가하면 자바가 기본으로 제공하던 기본 생성자가 사라집니다. 하지만 JPA 엔티티는 기본 생성자가 반드시 있어야 하므로, **@NoArgsConstructor** 어노테이션을 함께 붙여줍니다.

빌더 패턴(Builder Pattern)이란?

객체를 생성할 때 순서에 의존하지 않고, 어떤 필드에 어떤 값이 들어가는지 명시적으로 지정하는 패턴입니다.

@Builder를 사용하면 복잡한 매개변수 순서를 외울 필요 없이, 내가 원하는 필드의 값을 채워 넣을 수 있습니다.

3.1.3 요청 DTO 만들기

요청 DTO는 클라이언트가 전송한 데이터를 담는 클래스입니다. 클라이언트가 데이터를 보내면, 스프링이 요청 바디에 담긴 값들을 이름이 일치하는 DTO 필드에 자동으로 채워줍니다. 덕분에 컨트롤러는 복잡한 변환 과정 없이 이 DTO를 매개변수로 받아 바로 사용할 수 있습니다.

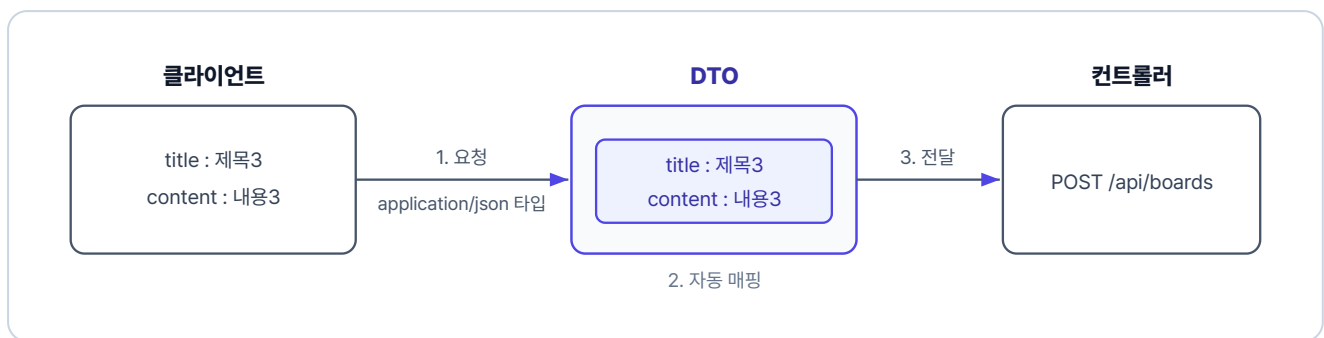


그림 3-3. 요청 데이터와 DTO

SaveDTO는 값만 담은 일반 객체라 영속 상태가 되지 못합니다. 그래서 DTO 안에 엔티티로 바꾸는 `toEntity()` 를 추가합니다.

`board/BoardRequest.java` 를 열어 아래와 같이 작성합니다.

실습 2 board/BoardRequest.java. 요청 DTO와 엔티티 변환

```
public class BoardRequest {
    public record SaveDTO(String title, String content) {

        // DTO를 엔티티로 변환한다
        public Board toEntity() {
            return Board.builder()
                .title(title)
                .content(content)
                .build();
        }
    }

    public record UpdateDTO(String title, String content) {
    }
}
```

```
}
```

`record`는 데이터 전달을 목적으로 하는 특수 클래스입니다. 필드만 선언하면 생성자와 getter 같은 메서드를 자동으로 만들어 줍니다.

3.2 응답 DTO

응답 DTO는 클라이언트에게 내보낼 값만 담는 클래스입니다. 서비스가 조회한 엔티티를 그대로 반환하지 않고, 이 클래스로 옮겨 담아 반환합니다.

`board/BoardResponse.java`를 열어 아래와 같이 작성합니다.

실습 3 board/BoardResponse.java. 응답 DTO

```
public class BoardResponse {  
  
    // 1. 목록용 DTO. 엔티티를 받아 보여줄 값만 담는다  
    public record DTO(Integer boardId, String title, String content) {  
  
        public DTO(Board board) {  
            this(board.getId(), board.getTitle(), board.getContent());  
        }  
    }  
  
    // 2. 상세용 DTO. 지금은 목록용과 같지만 앞으로 달라진다  
    public record DetailDTO(Integer boardId, String title, String content) {  
  
        public DetailDTO(Board board) {  
            this(board.getId(), board.getTitle(), board.getContent());  
        }  
    }  
}
```

응답 DTO는 기능마다 따로 만드는 것이 좋습니다. 다만 이 책에서는 편의를 위해 공통 DTO를 사용하고, 게시글 상세만 별도의 DTO를 만듭니다.

3.3 JpaRepository 적용

챕터 2의 리포지토리는 **EntityManager**를 주입받아 저장과 조회, 삭제 메서드를 직접 작성했습니다. 이 기본 동작들은 **JpaRepository**에 이미 들어 있어, 이 인터페이스를 상속하면 스프링이 대신 구현해 줍니다. 메서드를 적지 않아도 저장과 조회, 삭제를 호출할 수 있습니다.

`board/BoardRepository.java`를 열어 아래와 같이 변경합니다.

실습 4 board/BoardRepository.java. JpaRepository 상속

```
public interface BoardRepository extends JpaRepository<Board, Integer> {  
}
```

리포지토리는 클래스가 아니라 인터페이스로 선언합니다. `JpaRepository`를 상속할 때 다른 엔티티와 기본 키의 타입을 차례로 적습니다.

또한 조회 결과를 **Optional** 타입으로 반환할 수 있습니다. **Optional**은 값이 존재할 수도, 존재하지 않을 수도 있는 상태를 감싸는 래퍼(**Wrapper**) 클래스로, 주로 null로 인한 에러를 방지하기 위해 사용됩니다. 값이 없어도 null이 그대로 넘어가지 않으므로, 게시글을 찾지 못한 경우를 예외로 처리할 수 있습니다.

자주 쓰는 메서드는 다음과 같습니다.

메서드	하는 일
<code>save(엔티티)</code>	저장하고 저장된 엔티티를 반환합니다
<code>findById(기본 키)</code>	기본 키로 한 건을 조회해 Optional 에 담아 반환합니다
<code>findAll()</code>	전체를 List 로 반환합니다
<code>delete(엔티티)</code>	삭제합니다
<code>findBy필드명(값)</code>	직접 선언합니다. findBy 뒤 필드 이름을 보고 select문이 생성됩니다

3.4 예외 처리 추가

스프링에서 예외가 어떻게 처리되는지 흐름부터 알아보겠습니다.

3.4.1 예외 처리 흐름

클라이언트의 요청은 디스패처 서블릿을 통해 서비스 계층까지 전달됩니다. 만약 조회 결과가 없다면 서비스는 실패 사유를 담은 예외를 발생시킵니다. 예외가 발생하면 해당 메서드는 즉시 중단되고 그 아래 코드는 실행되지 않습니다.

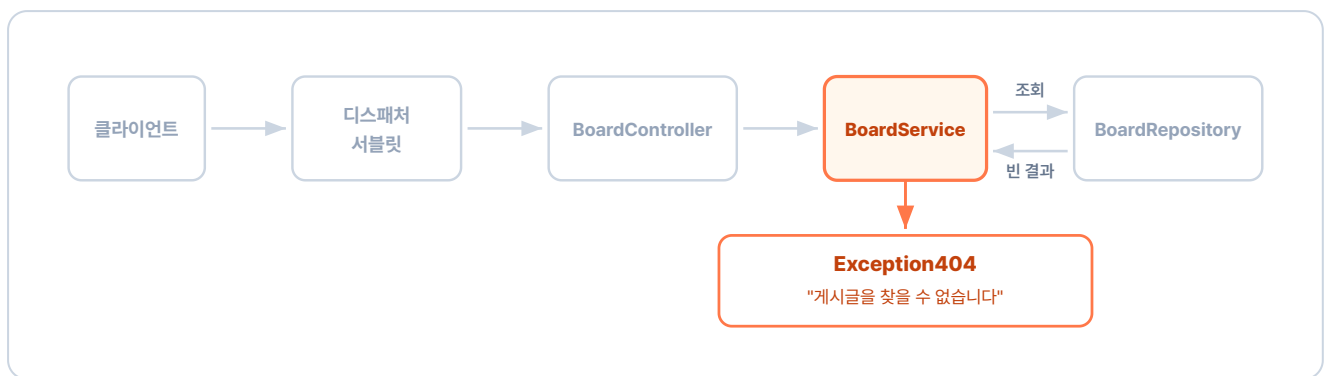


그림 3-4. 예외 발생 지점

중단된 메서드는 자신을 호출한 곳으로 예외를 넘깁니다. 넘겨받은 곳에서 다시 자신을 호출한 곳으로 넘기면서 예외는 디스패처 서블릿까지 전달됩니다.

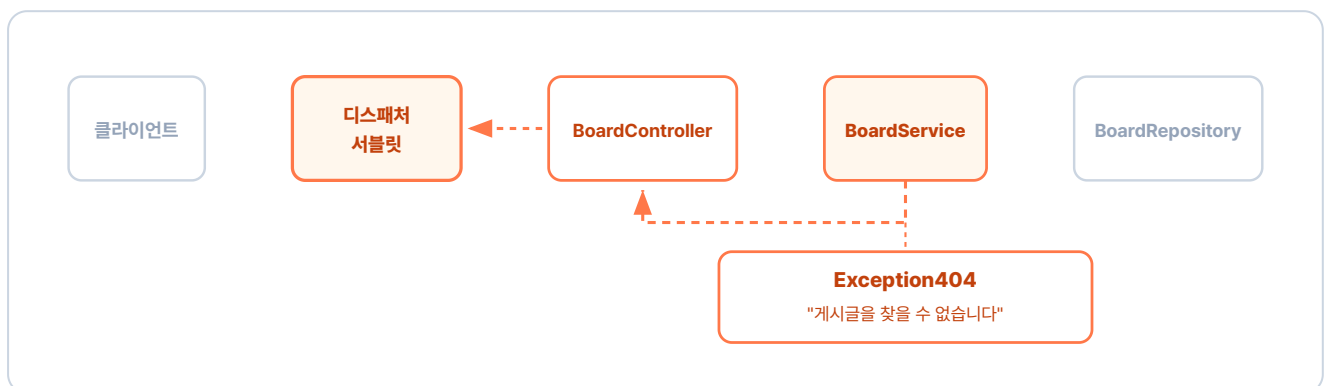


그림 3-5. 예외 전파 경로

디스패처 서블릿은 예외를 직접 처리하지 않습니다. 대신 발생한 예외의 타입에 맞는 메서드를 찾아 실행합니다. 이런 메서드들은 특정 컨트롤러에 속하지 않고 모든 컨트롤러에서 발생한 예외를 받으므로, 이들이 모인 클래스를 전역 예외 처리기라고 부릅니다.

실행된 메서드는 예외에 담긴 사유로 실패 응답을 만들어 반환합니다. 그 값이 디스패처 서블릿을 거쳐 클라이언트에게 응답으로 전달됩니다.

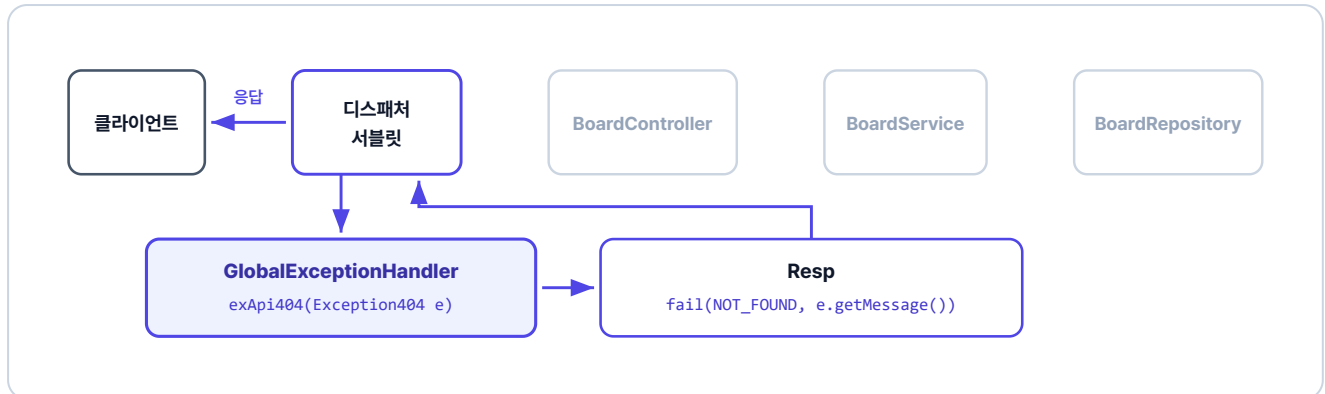


그림 3-6. 전역 예외 처리기의 응답

3.4.2 커스텀 예외 만들기

먼저 게시글을 찾지 못했을 때 발생시킬 예외 **Exception404**를 만듭니다. 서비스에서 이 예외를 발생시키면 전역 예외 처리기가 받아, 상태 코드 404와 실패 사유를 담은 응답을 클라이언트에게 반환합니다.

`core/handler/ex/Exception404.java`를 열어 아래와 같이 작성합니다.

실습 5 core/handler/ex/Exception404.java. 커스텀 예외

```

public class Exception404 extends RuntimeException {
    public Exception404(String message) {
        super(message);
    }
}
  
```

상황에 맞춰 직접 정의한 예외를 커스텀 예외라고 부릅니다. **RuntimeException**을 상속받아 구현하면, 예외가 발생하는 곳이나 메서드를 호출하는 곳 모두 `try-catch` 구문을 생략할 수 있습니다. 동일한 경로에 위치한 **Exception400**, **Exception401**, **Exception403**, **Exception500** 클래스 역시 이름만 다를 뿐 구조는 동일합니다.

이 책에서 사용하는 상태 코드는 다음과 같습니다.

상태 코드	뜻	예
400	처리할 수 없는 요청 값	제목 없이 게시글 작성

상태 코드	뜻	예
401	인증하지 않은 요청	로그인하지 않고 게시글 추가
403	인증했지만 권한이 없는 요청	다른 사람이 쓴 게시글 수정
404	찾을 수 없는 자원	없는 게시글 조회
500	서버 내부 오류	앞의 네 가지에 해당하지 않는 나머지 예외

3.4.3 전역 예외 처리

이제 앞에서 살펴본 전역 예외 처리기를 직접 만들어 보겠습니다. 클래스에 **@RestControllerAdvice**를 적용하면 그 클래스가 전역 예외 처리기가 됩니다. 이후 예외가 발생하면 디스패처 서블릿이 이 클래스 내부에서 예외 타입에 맞는 처리 메서드를 찾아 자동으로 실행합니다.

`core/handler/GlobalExceptionHandler.java`를 열어 아래와 같이 작성합니다.

실습 6 core/handler/GlobalExceptionHandler.java. 전역 예외 처리

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    // 1. 커스텀 예외는 저마다의 상태 코드로 바꾼다
    @ExceptionHandler(Exception400.class)
    public ResponseEntity<?> exApi400(Exception400 e) {
        return Resp.fail(HttpStatus.BAD_REQUEST, e.getMessage());
    }

    @ExceptionHandler(Exception401.class)
    public ResponseEntity<?> exApi401(Exception401 e) {
        return Resp.fail(HttpStatus.UNAUTHORIZED, e.getMessage());
    }

    @ExceptionHandler(Exception403.class)
    public ResponseEntity<?> exApi403(Exception403 e) {
        return Resp.fail(HttpStatus.FORBIDDEN, e.getMessage());
    }

    @ExceptionHandler(Exception404.class)
    public ResponseEntity<?> exApi404(Exception404 e) {
        return Resp.fail(HttpStatus.NOT_FOUND, e.getMessage());
    }
}
```

```

@ExceptionHandler(Exception500.class)
public ResponseEntity<?> exApi500(Exception500 e) {
    return Resp.fail(HttpStatus.INTERNAL_SERVER_ERROR, e.getMessage());
}

// 2. 나머지 모든 예외는 500으로 처리한다
@ExceptionHandler(Exception.class)
public ResponseEntity<?> exUnknown(Exception e) {
    return Resp.fail(HttpStatus.INTERNAL_SERVER_ERROR, "서버 오류가 발생했습니다");
}
}

```

3.5 서비스와 컨트롤러에 적용

3.5.1 서비스

이제 서비스에서 예외 처리를 해보겠습니다.

BoardService가 DTO로 값을 주고받고, 게시글이 없으면 예외를 발생시키도록 변경합니다.

`board/BoardService.java`를 열어 아래와 같이 작성합니다.

실습 7 board/BoardService.java. 주고받는 타입을 DTO로, 없으면 예외

```

public List<BoardResponse.DTO> 게시글목록() {
    return boardRepository.findAll().stream()
        .map(BoardResponse.DTO::new)
        .toList();
}

public BoardResponse.DetailDTO 게시글상세(Integer boardId) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    return new BoardResponse.DetailDTO(board);
}

@Transactional
public BoardResponse.DTO 게시글추가(BoardRequest.SaveDTO requestDTO) {
    Board savedBoard = boardRepository.save(requestDTO.toEntity()); // DTO -> 엔티티
    return new BoardResponse.DTO(savedBoard); // 저장된 게시글 반환
}

```

```

@Transactional
public BoardResponse.DTO 게시글수정(Integer boardId, BoardRequest.UpdateDTO request
DTO) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    // 데이터 체크
    board.setTitle(requestDTO.title());
    board.setContent(requestDTO.content());
    return new BoardResponse.DTO(board); // 수정된 게시글 반환
}

@Transactional
public void 게시글삭제(Integer boardId) {
    Board board = boardRepository.findById(boardId)
        .orElseThrow(() -> new Exception404("게시글을 찾을 수 없습니다"));
    boardRepository.delete(board);
}

```

리포지토리가 반환한 **Optional**에서 값을 꺼낼 때는 `orElseThrow()` 를 사용합니다. 이 메서드는 값이 있으면 게시글을 그대로 반환하고, 비어 있으면 인자로 받은 예외를 발생시킵니다.

3.5.2 컨트롤러

컨트롤러도 엔티티 대신 DTO로 값을 주고받도록 변경합니다.

`board/BoardController.java` 를 열어 아래와 같이 작성합니다.

실습 8 board/BoardController.java. 요청·응답 타입을 DTO로

```

@GetMapping
public ResponseEntity<?> findAll() {
    List<BoardResponse.DTO> respDTOList = boardService.게시글목록();
    return Resp.ok(respDTOList);
}

@GetMapping("/{boardId}")
public ResponseEntity<?> findById(@PathVariable("boardId") Integer boardId) {
    BoardResponse.DetailDTO respDTO = boardService.게시글상세(boardId);
    return Resp.ok(respDTO);
}

@PostMapping
public ResponseEntity<?> save(@RequestBody BoardRequest.SaveDTO requestDTO) {
    BoardResponse.DTO respDTO = boardService.게시글추가(requestDTO);
    return Resp.ok(respDTO);
}

```

```

@PutMapping("/{boardId}")
public ResponseEntity<?> update(@PathVariable("boardId") Integer boardId,
    @RequestBody BoardRequest.UpdateDTO requestDTO) {
    BoardResponse.DTO respDTO = boardService.게시글수정(boardId, requestDTO);
    return Resp.ok(respDTO);
}

@DeleteMapping("/{boardId}")
public ResponseEntity<?> deleteById(@PathVariable("boardId") Integer boardId) {
    boardService.게시글삭제(boardId);
    return Resp.ok(null);
}

```

이제 프로젝트를 실행해 결과를 확인해 보겠습니다. 먼저 1번 게시글을 조회하면 정상적인 데이터를 응답받습니다.

Hoppscotch 게시글 상세 조회

GET <http://localhost:8080/api/boards/1>

The screenshot shows the Hoppscotch interface for a GET request to `http://localhost:8080/api/boards/1`. The response status is 200 OK, with a response time of 293 ms and a size of 260 B. The response body is a JSON object:

```

{
  "status": 200,
  "msg": "성공",
  "body": {
    "boardId": 1,
    "title": "title1",
    "content": "content1"
  }
}

```

그림 3-7. 상세 조회 응답

다음으로 예외를 발생시켜 보겠습니다. 더미 데이터에 없는 999번 게시글을 조회해 응답 결과를 확인합니다.

Hoppscotch 없는 게시글 다시 조회

GET <http://localhost:8080/api/boards/999>

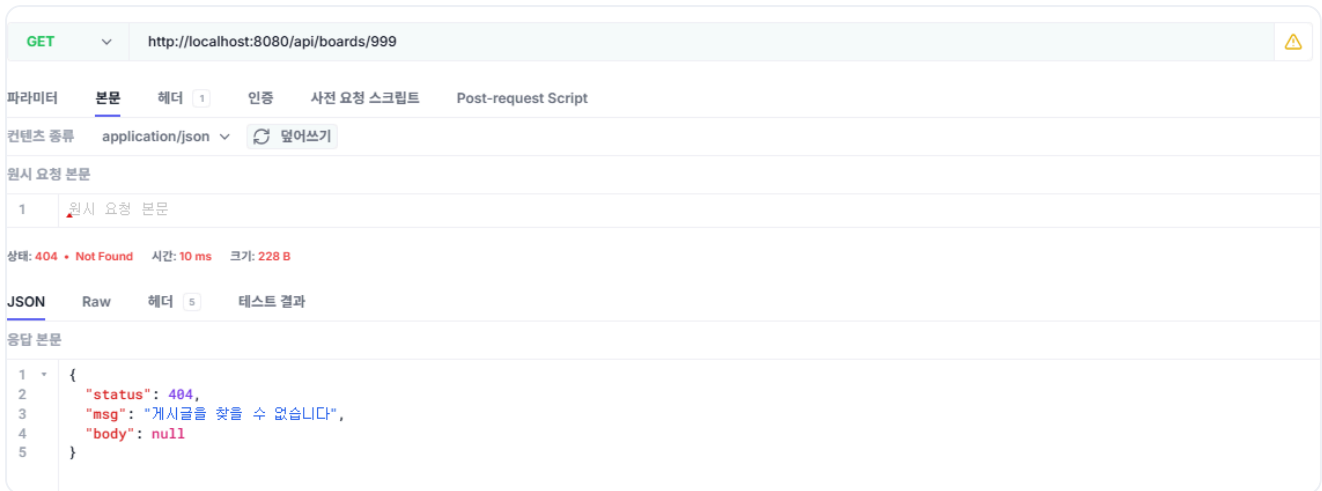


그림 3-8. 404 응답

챕터 2에서는 엔티티를 직접 요청과 응답에 사용했지만, 이번 챕터에서는 DTO를 활용해 데이터를 전달하는 구조로 개선했습니다. 또한 예외 처리를 추가하여, 처리 과정에서 문제가 발생하더라도 클라이언트가 상태 코드와 실패 사유를 응답받을 수 있습니다.

다음 챕터에서는 회원 엔티티를 추가해 게시글과 작성자를 연결하고, 이를 바탕으로 인증과 인가 기능을 구현해 보겠습니다.

이것만은 기억하자

- 엔티티를 요청과 응답에 직접 사용하지 않고 DTO로 주고받습니다. 테이블 구조가 바뀌어도 클라이언트와 주고받는 형식은 영향을 받지 않습니다.
- 요청이 실패하면 상태 코드와 사유를 담아 응답합니다. 서비스가 예외를 발생시키면 전역 예외 처리기가 한 곳에서 받아 상태 코드에 맞는 JSON으로 바꿉니다.